

(Scenario Based Questions)

1) A university is developing a student management system where each student has attributes like name, roll number, and courses enrolled. The system needs to allow students to perform actions such as enrolling in courses and checking grades. Question: Explain the relationship between objects, classes, data members, and methods in OOP with reference to this system.

==Concept: Objects, Classes, Data Members, Methods

- **Class:** A blueprint or template. In this case, a class Student can be created.
- **Objects:** Instances of the class. Each individual student (e.g., John, Priya) is an object of the Student class.
- **Data Members (Attributes):** These are variables inside the class. For example, name, rollNumber, and coursesEnrolled are data members of Student.
- **Methods (Functions):** Define behaviors of the object. Example methods:
 - enrollInCourse(String courseName)
 - checkGrades()

Explanation:

```
class Student {  
    String name;  
    int rollNumber;  
    List<String> coursesEnrolled;  
  
    void enrollInCourse(String course) {  
        coursesEnrolled.add(course);  
    }  
  
    void checkGrades() {  
        // logic to display grades  
    }  
}
```

2) A company is designing an employee management system. Some employee details, such as salary and bank details, should be hidden from other employees but accessible to HR. Other general details, like name and department, should be visible to everyone. Question: How does the private modifier differ from the protected modifier in OOP, and how can they be used to implement data security in this scenario?

==Employee Management System

Concept: Access Modifiers – private vs protected

- **private:** Accessible **only within the same class**. Used to hide sensitive data.
- **protected:** Accessible **within the same class, package, and subclasses**.

Implementation for Security:

```
class Employee {  
    public String name;  
    public String department;  
    private double salary;  
    private String bankDetails;  
  
    protected double getSalary() {  
        return salary;  
    }  
  
    protected String getBankDetails() {  
        return bankDetails;  
    }  
}
```

3) An e-commerce application allows users to apply different discount coupons. Some coupons apply a flat discount, while others apply a percentage-based discount. The system has a method apply Discount () that behaves differently based on the type of coupon. Question: Explain the difference between compile-time polymorphism and run-time polymorphism with reference to this system.

== Concept: Compile-Time vs Run-Time Polymorphism

- Compile-Time Polymorphism (Method Overloading):
Multiple methods with the same name but different parameters.

```
class Discount {  
    void applyDiscount(double amount) {  
        // flat discount logic  
    }  
  
    void applyDiscount(double amount, double percent) {  
        // percentage discount logic  
    }  
}
```

- Run-Time Polymorphism (Method Overriding):
A base class reference calls a method that is overridden in a subclass.

```
class Coupon {  
    void applyDiscount() {  
        System.out.println("Generic discount");  
    }  
}  
  
class FlatDiscount extends Coupon {  
    void applyDiscount() {  
        System.out.println("Flat discount applied");  
    }  
}  
  
class PercentageDiscount extends Coupon {  
    void applyDiscount() {  
        System.out.println("Percentage discount applied");  
    }  
}
```

```
}
```

```
// Runtime polymorphism
```

```
Coupon c = new FlatDiscount();
```

```
c.applyDiscount(); // Flat discount applied
```

4) A vehicle rental service provides different rental price calculations based on vehicle type. Cars have a base price per day, trucks have an additional weight-based charge, and motorcycles have a mileage-based price. Question: If a Vehicle class has an abstract method `calculate_rental_price ()`, how would different vehicle types (Car, Truck, Motorcycle) implement it?

```
==abstract class Vehicle {
```

```
    abstract double calculateRentalPrice(int days);
```

```
}
```

```
class Car extends Vehicle {
```

```
    double calculateRentalPrice(int days) {
```

```
        return days * 1000; // base price
```

```
    }
```

```
}
```

```
class Truck extends Vehicle {
```

```
    double calculateRentalPrice(int days) {
```

```
        return days * 1500 + 500; // additional weight charge
```

```
    }
```

```
}
```

```
class Motorcycle extends Vehicle {
```

```
    double calculateRentalPrice(int days) {
```

```
        return days * 500; // based on mileage
```

```
}  
}
```

5) A banking application needs to handle multiple types of exceptions, such as Insufficient Funds Exception, Invalid Account Exception, and Network Failure Exception. These exceptions can occur simultaneously when processing transactions. Question: How can you handle multiple exceptions in Java to ensure smooth transaction processing?

==Exception Handling (Multiple Catch Blocks or Multi-Catch)

Java allows handling multiple exceptions using either separate catch blocks or a multi-catch block.

```
try {  
    // transaction code  
} catch (InsufficientFundsException e) {  
    System.out.println("Not enough balance.");  
} catch (InvalidAccountException e) {  
    System.out.println("Account not found.");  
} catch (NetworkFailureException e) {  
    System.out.println("Network error. Please try again.");  
} catch (Exception e) {  
    System.out.println("An unexpected error occurred.");  
}
```

Alternative using multi-catch:

```
try {  
    // transaction code  
} catch (InsufficientFundsException | InvalidAccountException | NetworkFailureException e)  
{  
    System.out.println("Transaction failed: " + e.getMessage());  
}
```

6) Identify and Fix the Error in the Given Code

// Interface Animal

```

interface Animal {
    void makeSound();
}

// Interface Cat
interface Cat {
    void purr();
}

// Dog class implementing both interfaces
class Dog extends Animal, Cat {
    public void makeSound() {
        System.out.println("Dog barks!");
    }
    public void purr() {
        System.out.println("Dog cannot purr, but method must be implemented!");
    }
}

// Main class
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.makeSound();
        dog.purr();
    }
}

===

// Interface Animal
interface Animal {
    void makeSound();
}

```

```
}
```

```
// Interface Cat
```

```
interface Cat {
```

```
    void purr();
```

```
}
```

```
// Dog class implementing both interfaces
```

```
class Dog implements Animal, Cat {
```

```
    public void makeSound() {
```

```
        System.out.println("Dog barks!");
```

```
    }
```

```
    public void purr() {
```

```
        System.out.println("Dog cannot purr, but method must be implemented!");
```

```
    }
```

```
}
```

```
// Main class
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Dog dog = new Dog();
```

```
        dog.makeSound();
```

```
        dog.purr();
```

```
    }
```

```
}
```

7) A flight booking system allows customers to enter their age while booking a ticket. If the user enters a negative number, the system should throw an exception. However, missing a required field should cause a different exception. Question: Differentiate between checked and unchecked exceptions in Java and explain how they can be used to handle such cases in the flight booking system.

===

Feature	Checked Exceptions	Unchecked Exceptions
Definition	Exceptions that are checked at compile time.	Exceptions that are not checked at compile time (RuntimeExceptions).
Class Type	Subclasses of Exception but not RuntimeException.	Subclasses of RuntimeException.
Handling Requirement	Must be either caught using a try-catch block or declared using the throws keyword.	Do not need to be explicitly caught or declared.
Example	IOException, SQLException, FileNotFoundException.	NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException.
Cause	Usually due to external factors, such as file I/O, network failure, etc.	Usually caused by programming bugs, such as logic errors or invalid operations.
Checked by Compiler	Yes, the compiler checks that checked exceptions are handled properly.	No, the compiler does not check for unchecked exceptions.
Programmer Responsibility	The programmer must handle or declare these exceptions explicitly.	The programmer is not required to handle or declare unchecked exceptions, but it's often good practice to do so.
Example Code	throw new IOException();	throw new NullPointerException();

In the flight booking system:

- If a user enters a negative age, it would be a runtime issue, so it can be handled with an **unchecked exception** (like IllegalArgumentException).
- Missing a required field could be handled using a **checked exception** (like IOException) as it's a specific case that the programmer should anticipate and handle.

8) What will happen if an exception occurs and there is no catch block?

== If an exception occurs and there is no catch block, the program will terminate abnormally. The exception will propagate up the call stack, and if it reaches the main method without being caught, it will cause the program to exit, and an error message will be printed.

9) A software development company is working on an employee attendance system. Each employee has a unique ID and login time. The system initializes employee details when they log in and provides methods to retrieve and update their attendance data. Question: What is the difference between a method and a constructor in OOP? Explain with reference to the employee attendance system.

==

Feature	Method	Constructor
Definition	A method is a block of code that performs a specific task or operation.	A constructor is a special method used to initialize objects of a class.
Purpose	Used to define behaviors or actions of an object.	Used to initialize an object when it is created.
Name	Can have any name (except the name must match the method it is intended to perform).	The constructor name must be the same as the class name.
Return Type	Always has a return type (e.g., void, int, etc.).	Does not have a return type (not even void).
Invocation	Methods are called explicitly using the object or class.	Constructors are called implicitly when an object is created.
Parameters	Can accept any number of parameters or none.	Can accept parameters (called parameterized constructors) or none (default constructor).
Access Modifier	Can have different access modifiers (public, private, etc.).	Can have access modifiers, but typically is public for easy object creation.
Example	java public void display() { System.out.println("Hello"); }	java public MyClass() { this.value = 10; }

Feature	Method	Constructor
Execution	Executed when explicitly called.	Executed automatically when an object is created using new.
Inheritance	Methods can be inherited by subclasses (unless overridden).	Constructors are not inherited, but subclasses can call the superclass constructor using super().

10) A bank application stores customer account details securely. The account balance should not be directly accessible but can be retrieved or updated using specific methods. Question: Explain how getter and setter methods contribute to encapsulation. Provide a code example demonstrating how they can be used in this banking scenario.

==Encapsulation is the concept of hiding the internal details of an object and only exposing functionality through methods. Getter and setter methods are used to access and update the values of private fields.

Code Example (Bank Application):

```
class BankAccount {
    private double balance; // private field

    // Getter for balance
    public double getBalance() {
        return balance;
    }

    // Setter for balance
    public void setBalance(double balance) {
        if (balance >= 0) {
            this.balance = balance;
        } else {
            System.out.println("Invalid balance!");
        }
    }
}
```

```
}  
}  
}
```

11) A university wants to model its staff hierarchy. A Professor is a type of Employee, and a Department has multiple Professors. Question: Describe the "is-a" and "has-a" relationships in OOP with reference to this scenario. Provide an example for each.

=="Is-A" and "Has-A" Relationships in OOP:

- Is-A Relationship: This is a type of inheritance where one class is a type of another. For example, a Professor is an Employee — hence, a Professor "is a" type of Employee.
- Has-A Relationship: This represents composition or aggregation. For example, a Department has multiple Professors.

Examples:

- Is-A: A Professor is an Employee

```
class Employee {  
    // Employee details  
}  
  
class Professor extends Employee {  
    // Professor details  
}
```

Has-A: A Department has multiple Professors.

```
class Department {  
    private List<Professor> professors;  
}
```

12) A game development team is using both C++ and Java to design a game engine. They notice that Java requires interface and implements keywords, while C++ does not. Question: Why does C++ not require the interface or implements keyword like Java? Explain with a code example.

==C++ uses abstract classes to achieve similar functionality as Java's interfaces. In C++, a class can directly inherit from multiple classes (including abstract ones), without needing an interface or implements keyword.

Example in C++:

```
class Animal {  
  
public:  
  
    virtual void makeSound() = 0; // Pure virtual function  
};
```

```
class Dog : public Animal {  
  
public:  
  
    void makeSound() override {  
        std::cout << "Dog barks!" << std::endl;  
    }  
};
```

13) A social media platform is implementing user profiles. Some details, such as username and profile picture, are visible to everyone, while other details, like contact number and email, are hidden and can only be accessed via a request system. Question: What is the difference between Encapsulation and Abstraction in OOP? How would you use these concepts to design the privacy settings in this social media application?

==

Feature	Encapsulation	Abstraction
Definition	Encapsulation is the concept of bundling data (variables) and methods that operate on the data into a single unit, i.e., a class.	Abstraction is the concept of hiding the complex implementation details and showing only the essential features of an object.

Feature	Encapsulation	Abstraction
Focus	Focuses on data hiding and protecting the state of an object from outside interference and misuse.	Focuses on hiding complexity by providing a simple interface to interact with the object.
Visibility	Hides the internal state and requires all interactions to be performed through well-defined methods (getters and setters).	Hides the internal workings and only exposes relevant functionality to the user.
Implementation	Achieved by making class variables private and providing public getter and setter methods.	Achieved by using abstract classes or interfaces to define high-level behaviors.
Objective	To protect the object's state and ensure it is only changed in a controlled way.	To reduce complexity by hiding unnecessary details and focusing on essential aspects.
Example	A class with private fields and public getter/setter methods to access those fields.	An interface or abstract class that provides methods without detailing the implementation.
Example Code	<pre>java
class Employee {
 private String name;
 public String getName() { return name; }
 public void setName(String name) { this.name = name; }
}</pre>	<pre>java
abstract class Shape {
 abstract void draw();
}
class Circle extends Shape {
 void draw() { System.out.println("Drawing Circle"); }
}</pre>
Relation	Encapsulation deals with how data is stored and accessed.	Abstraction deals with what operations can be performed on the object.
Level of Operation	Operates at the class level by controlling the access to data and methods.	Operates at the interface or class design level by defining the behavior an object should have.
Complexity Handling	Manages internal complexity by restricting direct access to internal data.	Manages external complexity by providing a simplified interface to the user.

Example in Social Media Application:

- Encapsulation: User profile details like contact number and email are kept private using getter and setter methods.
- Abstraction: The system hides the complexity of how the data is fetched or displayed, exposing only a simple interface for users to request their profile details.

14) A student information system needs to store student records in a file. It should allow reading student details when needed and updating them efficiently. Question: Explain how to read and write files in C++ using fstream. Provide an example of how the student information system could implement this feature.

```
==#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
class Student {
```

```
public:
```

```
    std::string name;
```

```
    int age;
```

```
    void readFromFile() {
```

```
        std::ifstream file("students.txt");
```

```
        if (file.is_open()) {
```

```
            file >> name >> age;
```

```
            file.close();
```

```
        }
```

```
    }
```

```
    void writeToFile() {
```

```
        std::ofstream file("students.txt");
```

```
        if (file.is_open()) {
```

```
            file << name << " " << age << std::endl;
```

```

        file.close();
    }
}
};

int main() {
    Student student;
    student.name = "John";
    student.age = 20;
    student.writeToFile();
    student.readFromFile();
    std::cout << "Student: " << student.name << ", Age: " << student.age << std::endl;
}

```

15) Identify and Fix the Error in the Given Code

```

public class ExceptionExample {
    public static void main(String[] args) {
        try {
            int num = 5 / 0;
        }
        catch () {
            System.out.println("Cannot divide by zero");
        }
    }
}

===public class ExceptionExample {
    public static void main(String[] args) {
        try {
            int num = 5 / 0;

```

```

    }
    catch (ArithmeticException e) {
        System.out.println("Cannot divide by zero");
    }
}
}

```

16) A payment gateway processes multiple types of transactions. While executing a transaction, different exceptions may occur, such as Insufficient Balance Exception, Invalid Card Exception, or Transaction Time out Exception. The system must handle all these exceptions to ensure smooth transactions. Question: How can multiple exceptions be handled in Java to make sure the payment gateway runs without failure? Provide a suitable example.

=== In Java, multiple exceptions can be handled using multiple catch blocks or a multi-catch feature (since Java 7), which allows you to catch several exceptions in a single catch block. To ensure smooth transactions in a payment gateway, the system must handle specific exceptions like InsufficientBalanceException, InvalidCardException, and TransactionTimeoutException.

Example:

```

class PaymentGateway {

    public void processTransaction(double balance, String cardDetails) throws Exception {
        try {
            if (balance <= 0) {
                throw new InsufficientBalanceException("Insufficient balance in the account.");
            }

            if (cardDetails == null || cardDetails.isEmpty()) {
                throw new InvalidCardException("Card details are invalid.");
            }

            if (Math.random() > 0.95) { // Simulate a transaction timeout
                throw new TransactionTimeoutException("Transaction timed out.");
            }
        }
    }
}

```

```

    }

    System.out.println("Transaction successful!");
} catch (InsufficientBalanceException | InvalidCardException |
TransactionTimeoutException e) {
    System.out.println("Error: " + e.getMessage());
} catch (Exception e) {
    System.out.println("Unexpected error: " + e.getMessage());
}
}

```

// Custom exceptions

```

class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

```

```

class InvalidCardException extends Exception {
    public InvalidCardException(String message) {
        super(message);
    }
}

```

```

class TransactionTimeoutException extends Exception {
    public TransactionTimeoutException(String message) {
        super(message);
    }
}

```

```

public static void main(String[] args) {

```

```

PaymentGateway gateway = new PaymentGateway();

try {
    gateway.processTransaction(500, "Card123");
} catch (Exception e) {
    System.out.println("Main error handler: " + e.getMessage());
}
}
}

```

17) A hospital management system stores patient records. Each patient has a unique ID, name, and medical history. The system should allow doctors to create and access patient records efficiently. Question: What is the difference between a Class and an Object in OOP? Explain using this hospital management system as an example.

==

Feature	Class	Object
Definition	A class is a blueprint or template for creating objects.	An object is an instance of a class with actual values.
Purpose	Defines the properties (fields) and behaviors (methods) common to all objects of that type.	Represents a specific instance of a class.
Example	class Patient { int id; String name; String medicalHistory; }	Patient patient1 = new Patient(); patient1.id = 101;
Memory	A class does not occupy memory until instantiated.	An object occupies memory and holds actual data.
Existence	The class exists as a definition only, not as an entity.	The object exists as a real entity in memory during runtime.
Instantiation	A class is not instantiated directly, it's a template.	An object is created by calling the class's constructor.

Example in Hospital Management System:

- Class: Patient class defines the blueprint with properties like id, name, and medicalHistory.

- Object: A specific patient like patient1 is an object of the Patient class with actual data.

18) A company is developing a role-based access control system where different users have different levels of access. Admin can modify and delete data, Manager can view and edit, while Employees can only view certain details. Question: Create a comparison table of access modifiers (private, protected, public, and default) and explain how they can be used in this system to restrict access appropriately.

==

Access Modifier	Description	Visibility	Usage Example
private	The field or method is only accessible within the same class.	Only within the class where it is defined.	private String password;
protected	The field or method is accessible within the same package or subclasses.	Within the same package and subclasses (including those in other packages).	protected String accessLevel;
public	The field or method is accessible from anywhere.	Everywhere (within the same class, package, and other packages).	public void deleteData();
default (no modifier)	The field or method is accessible within the same package only.	Only within the same package.	String departmentName; (no modifier)

Example in Role-Based Access Control System:

- Admin (can modify and delete data) - Use public for methods like deleteData().
- Manager (can view and edit) - Use protected for methods like editData().
- Employee (can only view data) - Use default for methods like viewData().

19) A robotics startup is using C++ and Java to develop a robotic arm control system. While coding the interfaces for different sensors, they observe that Java requires interface and implements keywords, whereas C++ does not. Question: Why does C++ not require the interface or implements keyword like Java? Explain with a code example.

==In Java, interfaces are explicitly defined using the interface keyword, and classes implement them using the implements keyword. In contrast, C++ does not have a distinct interface keyword because it uses abstract classes to achieve similar functionality.

Example in C++ (without interface or implements):

```
#include <iostream>
```

```
using namespace std;
```

```
class Sensor {
```

```
public:
```

```
    virtual void readData() = 0; // Pure virtual function (abstract method)
```

```
};
```

```
class TemperatureSensor : public Sensor {
```

```
public:
```

```
    void readData() override {
```

```
        cout << "Reading temperature data." << endl;
```

```
    }
```

```
};
```

```
int main() {
```

```
    Sensor* sensor = new TemperatureSensor();
```

```
    sensor->readData(); // Calls overridden method
```

```
    delete sensor;
```

```
    return 0;
```

```
}
```

20) A school management system has a Student class that inherits from the Person class. The Person class has a method getDetails(), which returns basic information, but the Student class needs to provide additional student specific details. Question: How do the super and this keywords work in Java? Explain their use in this school management system.

==

- **this:** Refers to the current instance of the class. It is used to differentiate between instance variables and parameters with the same name or to call another constructor in the same class.
- **super:** Refers to the superclass (parent class) of the current object. It is used to call the parent class's constructor or methods.

Example:

```
class Person {  
    String name;  
  
    public Person(String name) {  
        this.name = name; // 'this' refers to the current class's instance variable  
    }  
  
    public void getDetails() {  
        System.out.println("Name: " + name);  
    }  
}  
  
class Student extends Person {  
    int studentId;  
  
    public Student(String name, int studentId) {  
        super(name); // 'super' calls the constructor of the superclass (Person)  
        this.studentId = studentId; // 'this' refers to the current class's instance variable  
    }  
}
```

```

@Override

public void getDetails() {

    super.getDetails(); // Calls the getDetails method from Person class

    System.out.println("Student ID: " + studentId);

}

}

```

```

public class Main {

    public static void main(String[] args) {

        Student student = new Student("John", 12345);

        student.getDetails(); // Calls the overridden method in Student class

    }

}

```

21) A vehicle rental management system allows customers to rent vehicles. Customers can specify rental duration, vehicle type, or even add extra services like insurance. Question: How would you implement method overloading for a `rent_vehicle()` method to support different customer needs? Provide an example.

==Method overloading in Java allows you to define multiple methods with the same name but different parameter lists. For a vehicle rental management system, you can use method overloading to support different customer needs such as specifying rental duration, vehicle type, or adding extra services like insurance.

Example:

```

class VehicleRental {

    // Method to rent a vehicle with rental duration only

    public void rent_vehicle(int duration) {

        System.out.println("Vehicle rented for " + duration + " days.");

    }

}

```

```
// Method to rent a vehicle with rental duration and vehicle type
public void rent_vehicle(int duration, String vehicleType) {
    System.out.println("Vehicle rented for " + duration + " days. Vehicle type: " +
vehicleType);
}

// Method to rent a vehicle with rental duration, vehicle type, and extra services like
insurance
public void rent_vehicle(int duration, String vehicleType, boolean insurance) {
    String insuranceStatus = insurance ? "with insurance" : "without insurance";
    System.out.println("Vehicle rented for " + duration + " days. Vehicle type: " +
vehicleType + ". " + insuranceStatus);
}

public static void main(String[] args) {
    VehicleRental rental = new VehicleRental();
    rental.rent_vehicle(5); // Rent vehicle for 5 days
    rental.rent_vehicle(3, "SUV"); // Rent SUV for 3 days
    rental.rent_vehicle(7, "Sedan", true); // Rent Sedan for 7 days with insurance
}
}
```

22) A banking application processes transactions. If an error occurs, such as insufficient funds or invalid account number, the system should handle it gracefully instead of crashing. Question: Define Exception Handling and Error Handling and explain how they help in building reliable banking applications.

=== Exception Handling:

- In Java, exceptions are used to handle situations that might occur during the execution of a program. Exceptions are conditions that a program can anticipate and recover from.
- They represent runtime issues like invalid input, network errors, database issues, etc.
- Try-catch blocks are used to handle exceptions.

- Example: Insufficient funds or invalid account numbers in a banking application can throw exceptions that can be caught and handled to prevent application crashes.
- Error Handling:
 - Errors represent serious problems that a program cannot typically recover from (e.g., out of memory, JVM errors, etc.).
 - Errors are usually handled by the runtime environment rather than the program itself.
 - In general, errors are not caught by regular exception handling (using try-catch blocks).
 - Example: An OutOfMemoryError would indicate a serious issue that needs to be addressed by the system or environment, not by handling it in code.

How They Help in Banking Applications:

- Exception Handling: Ensures that the application remains robust and doesn't crash when common issues (like insufficient funds) occur. For example, the system could prompt the user to check their balance when an insufficient funds exception is caught.
- Error Handling: Helps to identify and log critical errors such as database or server crashes that might require system-level intervention.

23) Identify and Fix the Error in the Given Code

```
class Student {
    private int age = 20;
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        System.out.println(s.age);
    }
}
```

```

=== class Student {
    private int age = 20;

    // Provide a public getter method to access the age
    public int getAge() {
        return age;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        System.out.println(s.getAge()); // Use the getter method to access age
    }
}

```

24) A flight booking system allows customers to book tickets. If an invalid seat number is entered, the system should immediately generate an exception. However, some exceptions, like invalid passport numbers, should be declared in the method signature for the calling function to handle. Question: What is the difference between throw and throws in Java? Explain with reference to this flight booking system.

==

Feature	throw	throws
Definition	throw is used to explicitly throw an exception in the body of a method or block of code.	throws is used in a method signature to declare that a method might throw one or more exceptions.
Usage	Used to throw an exception from a method or block of code.	Used to declare that a method may throw exceptions to be handled by the caller.

Feature	throw	throws
Where Used	Used inside a method or block of code.	Used in the method signature (e.g., public void method() throws Exception).
Type of Exceptions	Can throw both checked and unchecked exceptions.	Typically used for checked exceptions (exceptions that are checked at compile time).
Syntax	throw new Exception("Message");	public void method() throws Exception;
Responsibility	The method or block that uses throw is responsible for throwing the exception.	The method that declares throws is indicating that the caller should handle the exception.
Example	java throw new IllegalArgumentException("Invalid input!");	java public void process() throws IOException { // code }
Effect	Immediately terminates the current method and transfers control to the nearest catch block (if any) or stops execution.	Allows the exception to propagate up the call stack, requiring the caller to handle it.

```
class FlightBooking {
```

```
    // Method throws an exception for invalid passport number
```

```
    public void bookTicket(String passportNumber) throws InvalidPassportException {
```

```
        if (passportNumber == null || passportNumber.length() != 9) {
```

```
            throw new InvalidPassportException("Invalid passport number.");
```

```
        }
```

```
        System.out.println("Ticket booked successfully!");
```

```
    }
```

```
    // Method to check seat number (throws exception if invalid seat number is entered)
```

```
public void checkSeat(int seatNumber) {  
    if (seatNumber < 1 || seatNumber > 100) {  
        throw new IllegalArgumentException("Invalid seat number.");  
    }  
    System.out.println("Seat " + seatNumber + " is available.");  
}
```

```
public static void main(String[] args) {  
    FlightBooking booking = new FlightBooking();  
    try {  
        booking.bookTicket("12345"); // Throws InvalidPassportException  
    } catch (InvalidPassportException e) {  
        System.out.println("Error: " + e.getMessage());  
    }  
  
    booking.checkSeat(150); // Throws IllegalArgumentException  
}
```

```
// Custom Exception for invalid passport  
class InvalidPassportException extends Exception {  
    public InvalidPassportException(String message) {  
        super(message);  
    }  
}
```